The Walking Packages: A Survival Analysis of ROS Repositories

Juliana Freitas[1], Elijah Phifer[1] and Felipe Fronchetti[1]

[1]*Louisiana State University, Baton Rouge, Louisiana, USA*

Abstract

The Robot Operating System (ROS) serves as a primary foundation for system development in the robotics domain [1]. Its long-term popularity is often correlated with the development of its packages, maintained by internal and external contributors in open-source software (OSS) repositories [2]. While ROS packages are considered a vital part of ROS, many cease activity for a still unexplored set of circumstances. Previous research on OSS development has shown that the lifecycle of open-source projects is often unstable, with many repositories transitioning through three states: active, zombie (where superficial contributions continue but do not meaningfully advance the software), and inactive [3]. In an attempt to better understand why ROS packages become inactive (or turn into walking zombies), we present a work-in-progress survival analysis of repositories in the ROS package ecosystem. Using data extracted from 461 ROS package repositories on GitHub, we examine repository survivability across two dimensions: survival rates over time, and repository characteristics associated with long-term survival. Our preliminary findings suggest that...**[complete]**

Keywords

Survival Analysis, Software Repositories, Packages, ROS, OSS

1. Introduction

The Robot Operating System (ROS) has become a standard middleware for modern robotics systems, [4]. Its long-term success is closely related to the collaborative nature of the ROS development model, in which developers rely on existing packages to implement complex features such as perception, navigation, and the motion planning of robots [5]. Its modular publish-subscribe architecture and extensive ecosystem of open-source packages allow developers to easily encapsulate useful functionality into systems that can be distributed and integrated across a wide variety of robotic platforms [6].

This same reliance on packages introduces significant sustainability risks for software applications within the domain. The development of ROS packages, like other open-source software (OSS) projects, is largely driven by external developers and organizations, many of which receive *voluntary* contributions through their software repositories. As a result, these projects are particularly vulnerable to maintainer turnover [7, 8]. Packages that do not receive new contributions tend to become inactive, creating a maintenance gap on existing applications. In software engineering, the phenomenon of dependency abandonment is a well-known threat [3], but in the robotics domain, the consequences can be particularly severe [9]. Robotic applications are concentrated on complex cyber-physical systems that interact directly with the real world [10, 5]. Reliance on abandoned or outdated packages can lead to unpredictable robot behavior, compatibility issues with other packages or newer ROS distributions, and critical cybersecurity threats [5].

According to literature, when traditional OSS repositories lose core contributors, the project rarely dies immediately. Instead, repositories enter a "*zombie*" state, in which only superficial, non-code changes are made. Recent empirical studies on OSS survival rates describe these "*zombies*" as repositories that still exhibit some activity, such as issues, comments, or automated alerts, but lack meaningful human contributions [3]. Within the ROS ecosystem, the presence of "*walking packages*" can pose a significant barrier to software reuse. Roboticists often invest considerable time configuring and integrating packages into their systems. If they later discover that these packages are unmaintained

RoSE'26: 8th International Workshop on Robotics Software Engineering, June 1st, 2026, Vienna, Austria.
jfreit4@lsu.edu (J. Freitas); ephife3@lsu.edu (E. Phifer); ffronchetti@lsu.edu (F. Fronchetti)
© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

or incompatible with their current ROS distributions, the result can be delays and rework in the development process [5].

A survival analysis of ROS packages becomes even more relevant in the current context, following the end of life of ROS1 in 2025 and the ongoing transition of many projects to ROS2, a process that may leave behind a growing number of obsolete or unmaintained packages [11]. While previous research has investigated the quality of ROS repositories and the barriers to software reuse in robotics [5, 9], the long-term survival dynamics of ROS-specific software remains unexplored. In the broader context of OSS development, survival analysis has proven to be an effective method for modeling repository longevity and identifying risk factors associated with projects survivability [3, 12]. To address this research gap in the robotics domain, we propose a preliminary investigation on the survival dynamics of packages within the ROS ecosystem. Using contribution activity extracted from 462 ROS package repositories hosted on GitHub, we perform a preliminar survival analysis to answer three research questions:

RQ$_1$ *What is the survival rate of ROS package repositories?*

RQ$_2$ *How do the survival states of ROS package repositories change over time?*

RQ$_3$ *Which repository characteristics are associated with package inactivity?*

2. Related Work

Software Engineering in the ROS Ecosystem: Although little is known about the survival rate of package repositories in ROS, researchers have already explored software engineering practices and patterns in its ecosystem. For instance, Kolak et al. [11] examined how the need for robotics-specific knowledge influences collaboration among OSS contributors in ROS-based repositories. In this study, the authors found that the development of the most used ROS packages belong to a small, centralized cluster of foundational working groups, each organized around a different domain of expertise (e.g., research, industry). They show that the ROS ecosystem follows an organizational structure that is not commonly seen in other OSS ecosystems, with a strong reliance of small groups of experts. In a similar landscape, Pichler [9] shows that our goal with this preliminary investigation is to extend prior work on software engineering in ROS by providing an initial investigation of repository activity and the factors contributing to inactivity.

Survival Analysis in Open-Source Software: Survival analysis is a statistical technique used to study the time until an event is expected to occur [13]. Due to the nature of the data used in survival analysis, it is considered a special case of time-series analysis [3]. Survival analysis is used in open-source software (OSS) research to model project activity; existing literature suggests a significant number of projects become inactive within their first year [3]. The survival rate of open-source projects interests both the industries relying on OSS and the contributors deciding which projects to join. The survival of OSS projects has major implications for the sustainability and growth of these industries that rely on OSS [2, 14]. Our work extends previous research by investigating the survival rate of projects in the ROS ecosystem. Because ROS is a *standard* [1] in robotics, the sustainability of this ecosystem is vital. Thus, it serves as a valuable research focus to understand the state of software for robotics.

3. Method

3.1. Repository Selection and Activity Extraction

To define a clear scope of ROS-related repositories to analyze in our study, we limited our data extraction process to active ROS2 packages hosted on GitHub. By *active*, we refer to packages from ROS2

distributions that have not yet reached their end-of-life (EOL), which at the time of data extraction[1] included the Kilted, Jazzy, and Humble distributions. Packages are the fundamental unit of organization, build, and release for software in the ROS ecosystem. They can be developed by anyone, from ROS maintainers to external organizations, and therefore provide an appropriate setting for analyzing newcomer-related activities in the ROS ecosystem.

To identify these packages, we used ROS Index,[2] the official search engine for ROS resources. In total, 6,242 package–distribution entries were extracted across the three active distributions, corresponding to 2,660 unique packages hosted in 874 GitHub repositories. A single repository may host multiple ROS packages — 287 repositories (35.4%) do so, and the same repository often targets multiple distributions simultaneously. To ensure a minimum observation period and exclude non-code repositories, we removed repositories that were: (a) archived, (b) forks of other repositories, (c) less than six months old, or (d) lacking any detected programming language. This process removed 64 repositories (4 archived, 25 forks, 24 too recent, 15 with no languages), yielding a final dataset of **810 unique GitHub repositories** containing 2,250 unique packages.

3.2. Survival Analysis

To support the survival analysis, we constructed structured event tables from GitHub activity logs. For each repository, we collected full activity histories via the GitHub REST API, capturing five event types: *commits*, *issues*, *pull requests*, *issue comments*, and *pull-request reviews*. Each event record includes the repository identifier, the event timestamp, and the account type of the author, allowing us to reconstruct the temporal activity history of each project. Account types are determined using the API's native `type` field: *Human* (`type = "User"`), *Bot* (`type = "Bot"`, e.g., Dependabot), or *Organization* (`type = "Organization"`). This classification is used throughout the analysis: in the state machine (RQ_2), only commits authored by *Human* accounts qualify a month as *Running*.

Inactivity definition. Following [3], we classify a repository as inactive when it shows no recorded activity for more than 180 consecutive days before the study end date. Activity includes commits, issues, pull requests, issue comments, and pull-request reviews. A repository therefore becomes inactive only when all forms of engagement cease. Projects without human commits but with non-coding activity (e.g., issues or comments) correspond to the "Zombie" state defined by Ait et al. [3]; these repositories are considered active within the 180-day window and are treated as right-censored observations. The time-to-event is measured from repository creation to the date of the last recorded activity (for inactive repositories) or to the study end date (for active ones). Of the 810 repositories, 226 (27.9%) are classified as inactive and 584 (72.1%) remain active at the study end date.

Survival rate. We estimate the survival function $S(t)$, defined as the probability that a repository remains active beyond time t, using the *Kaplan-Meier* (KM) estimator [13] implemented in the `scikit-survival` library. The KM estimator accommodates right-censored observations without distributional assumptions. We compute $S(t)$ for the full dataset and separately for contributor tiers defined by the interquartile range (IQR) of contributor counts (Q1 = 3, Q3 = 18): Tier 1 ≤ 3, Tier 2 $\in [4, 18]$, Tier 3 > 18 contributors). Statistical differences between groups are assessed using the log-rank test.

State evolution. States are defined according to the type of activity observed in that month, following Ait et al. [3]: *Running* (at least one human commit), *Zombie* (non-code activity such as issue interactions, pull-request comments, or reviews, or bot activity, but no human commits), and *Dead* (no recorded activity of any kind). These monthly observations are used to build empirical transition matrices, from which we estimate month-to-month transition probabilities (e.g., Running→Zombie) and average state durations.

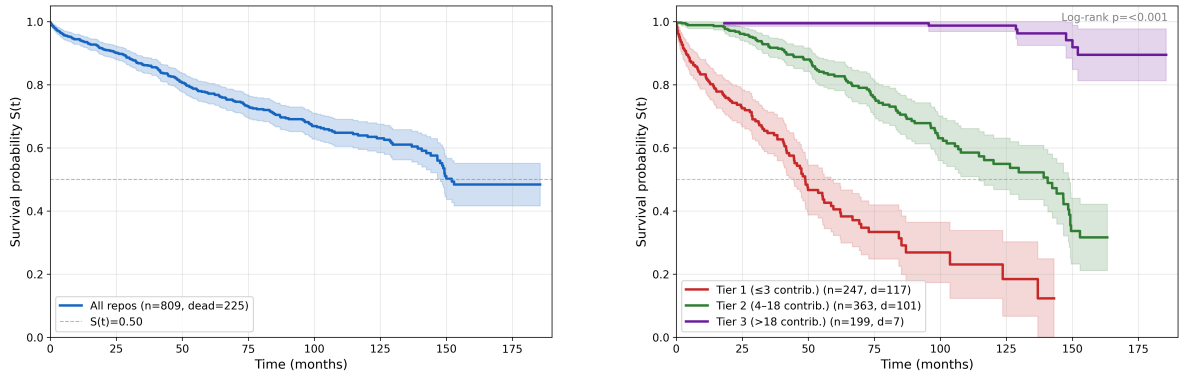
Inactivity factors. While the Kaplan-Meier estimator provides a visual and univariable representation of survival,We fit a Cox Proportional Hazards (CoxPH) model [13] to identify repository characteristics associated with inactivity. To avoid reverse-causality bias—whereby projects that survive longer naturally accumulate more raw commits and forks—we use activity *rates* normalized by project

[1] Data extraction: March 2026.
[2] https://index.ros.org/

age as covariates: commits per month, newcomers per month, forks per month, and stars per month [12]. These are supplemented by owner type (organization vs. user) and the presence of governance artifacts (`README`, `CONTRIBUTING`, `PULL_REQUEST_TEMPLATE`, `ISSUE_TEMPLATE`, and `CODE_OF_CONDUCT` files). The model is estimated with L2 regularization ($\alpha = 0.01$) to stabilize rare features, and evaluated on a stratified held-out test set (25% of repositories). Hazard ratios (HR) greater than 1 indicate increased risk of inactivity. We additionally train a Random Survival Forest (RSF) [3] to rank feature importance independently of CoxPH linearity assumptions, using permutation importance on the held-out test set.

4. Preliminary Results

(RQ1) What is the survival rate of ROS package repositories?



(a) Kaplan-Meier estimations of the survival curve considered over all ROS projects.

(b) Kaplan-Meier estimations of the survival curve of ROS projects grouped by the number of contributors in a project repository.

Figure 1: The probability of survival over time.

In this research question, we analyze the survival probability over time for all ROS repositories in our study. Figure 1aa illustrates this probability across the entire dataset. As shown, ROS repositories exhibit a gradual but continuous decline in activity over time. The overall survival rate is high: 77.1% of repositories remain active at the five-year mark, and only 226 (27.9%) became inactive by the study end date. The median survival time exceeds the observation window, meaning that most packages never fully transition to inactivity during the study period. These results compare favorably to broader OSS ecosystems—Ait et al. [3] reported a 5-year survival rate of 73% for general GitHub repositories—suggesting that ROS packages benefit from stronger institutional and community support.

Figure 1b extends this analysis by plotting the survival rate over time categorized by the number of contributors. Projects are grouped into three tiers based on the IQR of contributor counts in our dataset: Tier 1 ($\leq$ 3 contributors), Tier 2 (4–18 contributors), and Tier 3 ($>$ 18 contributors). Tier 1 projects exhibit significantly shorter lifespans: their survival probability drops to 50.7% by month 48 and falls further to 40.4% by month 60, crossing below the 50% threshold within the observation window. In contrast, Tier 2 and Tier 3 projects demonstrate much stronger resilience, reaching $S(60\text{m}) = 0.827$ and $S(60\text{m}) = 0.995$, respectively, without ever dropping below 50%. These findings indicate that projects with more contributors survive significantly longer—a pattern consistently reported in OSS research [3, 15]. Repositories that fail to attract new contributors tend to stagnate and become inactive, underscoring the critical role of onboarding and community growth in ensuring long-term package sustainability [15, 16].

(RQ2, RQ3) How do the survival states of ROS package repositories change over time, and which repository characteristics are associated with package inactivity?

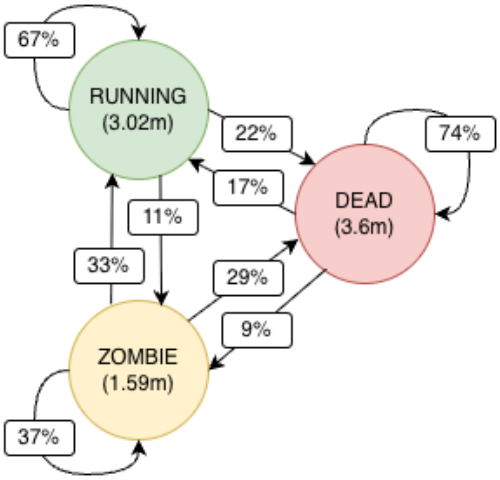


Figure 2: State machine for all repositories.

Table 1: Significant CoxPH predictors of repository inactivity. Bootstrap 95% CI.

Feature	HR	95% CI	p
Protective (HR < 1)			
Has Issue Template	0.024	0.013–0.075	$< 0.001^{***}$
Community Size	0.226	0.162–0.296	$< 0.001^{***}$
Forks/month	0.289	0.148–0.487	$< 0.001^{***}$
Commits/month	0.544	0.373–0.760	$< 0.001^{***}$
Is Organization	0.671	0.463–0.950	0.036^{*}
Risk (HR > 1)			
Newcomers/month	3.378	2.396–5.347	$< 0.001^{***}$
Has Newcomer Labels	4.121	2.813–7.111	$< 0.001^{***}$

For this research questions, we modeled the activity evolution of ROS repositories using the state machine shown in Figure **??**, adopting the methodology established by Ait et al. [3]. On average, repositories spend 3.02 months in the *Running* state before transitioning: most remain active in the following month (66.86%), while 11.54% enter a *Zombie* state and 21.60% become directly inactive. The *Zombie* state is notably transient, lasting only 1.59 months on average, and represents a critical window for recovery: about one-third of zombie repositories (32.98%) return to an active state, while 29.39% become fully inactive and 37.63% remain in limbo. Once a repository enters the *Inactive* state, the outcome tends to be persistent—73.84% of inactive repository-months are followed by continued inactivity—yet a non-trivial 17.09% manage to recover to an active state, suggesting that some repositories undergo temporary dormancy rather than permanent abandonment. This aligns with findings from Avelino et al. [17], who showed that OSS project abandonment is often gradual and sometimes reversible, particularly when institutional backing is present.

To understand the drivers behind these transitions, we analyzed the features that contribute most to project inactivity. Table 1 and Figure 3 identify the repository characteristics most strongly associated with package inactivity, connecting the patterns observed in the state machine with measurable repository attributes. The CoxPH model achieves a C-index of 0.864 on the held-out test set, and permutation importance confirms that community size is the single most predictive feature ($\Delta C = 0.220$), consistent with the tier divergence observed in RQ_1. Repositories with larger communities are substantially less likely to become inactive (HR = 0.226, $p < 0.001$)—their broader contributor base provides a buffer against the rapid transitions into zombie and inactive states observed in smaller projects. Fork rate (HR = 0.289, $\Delta C = 0.065$) and commit rate (HR = 0.544) further reinforce this picture: packages that are actively reused by other projects generate downstream dependencies that sustain maintenance effort, while regular commit activity keeps repositories out of the zombie state. Organizational ownership also offers protection (HR = 0.671, $p = 0.036$), consistent with findings by Ait et al. [3] that institution-backed projects outlast individually maintained ones. Issue templates emerge as a strong protective signal (HR = 0.024), suggesting that repositories with structured contribution workflows are better equipped to retain contributors over time. Conversely, newcomer-friendly labels are associated with higher inactivity risk (HR = 4.121, $\Delta C = 0.035$) even after controlling for community size—a pattern we hypothesize reflects *reactive* adoption: labels added by projects already struggling to sustain activity, rather than by thriving communities [18]. Taken together, these findings suggest that what keeps ROS packages alive is not documentation or signaling, but a stable community and sustained engagement [16].

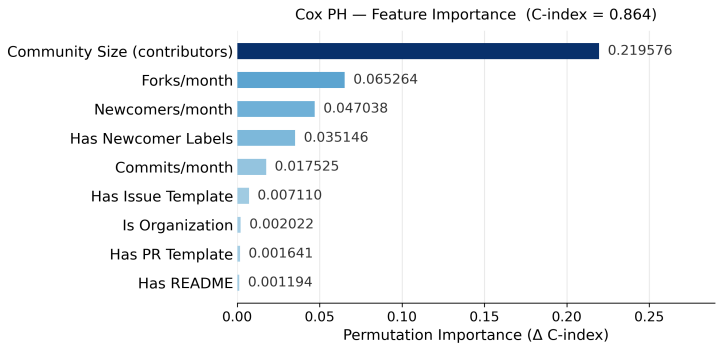


Figure 3: CoxPH permutation feature importance on the held-out test set (C-index = 0.864). Bar length shows mean decrease in C-index when each feature is shuffled; darker bars indicate higher importance.

References

[1] M. Albonico, M. Đorđević, E. Hamer, I. Malavolta, Software engineering research on the Robot Operating System: A systematic mapping study, Journal of Systems and Software 197 (2023).

[2] K. Crowston, J. Howison, Assessing the health of open source communities, Computer 39 (2006) 89–91.

[3] A. Ait, J. L. C. Izquierdo, J. Cabot, An empirical study on the survival rate of github projects, in: Proceedings of the 19th International Conference on Mining Software Repositories, MSR '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 365–375.

[4] I. Malavolta, G. Lewis, B. Schmerl, P. Lago, D. Garlan, How do you architect your robots? state of the practice and guidelines for ros-based systems, in: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice, 2020, pp. 31–40.

[5] P. Estefo, J. Simmonds, R. Robbes, J. Fabry, The robot operating system: Package reuse and community dynamics, Journal of Systems and Software 151 (2019) 226–242.

[6] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, A. Y. Ng, Sharing software with ros, IEEE Robotics Automation Magazine 17 (2009) 14–15.

[7] C. Miller, C. Kästner, B. Vasilescu, "we feel like we're winging it:" a study on navigating open-source dependency abandonment, in: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2023, pp. 1281–1293.

[8] I. Steinmacher, View Profile, T. U. Conte, View Profile, C. Treude, View Profile, M. A. Gerosa, View Profile, Overcoming open source project entry barriers with a portal for newcomers, in: Proceedings of the 38th International Conference on Software Engineering, ACM Conferences, 2016, pp. 273–284.

[9] M. Pichler, B. Dieber, M. Pinzger, Can i depend on you? mapping the dependency and quality landscape of ros packages, in: 2019 third IEEE international conference on robotic computing (IRC), IEEE, 2019, pp. 78–85.

[10] S. García, D. Strüber, D. Brugali, T. Berger, P. Pelliccione, Robotics software engineering: a perspective from the service robotics domain, in: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2020, Association for Computing Machinery, New York, NY, USA, 2020, p. 593–604.

[11] S. Kolak, A. Afzal, C. Le Goues, M. Hilton, C. S. Timperley, It takes a village to build a robot: An empirical study of the ros ecosystem, in: 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), IEEE, 2020, pp. 430–440.

[12] D. Robinson, K. Enns, N. Koulecar, M. Sihag, Two approaches to survival analysis of open source python projects, in: Proceedings of the 30th IEEE/ACM International Conference on Program

Comprehension, ICPC '22, Association for Computing Machinery, New York, NY, USA, 2022, p. 660–669.

[13] D. G. Kleinbaum, M. Klein, Survival analysis a self-learning text, Springer, 1996.

[14] J. Marsan, M. Templier, P. Marois, B. Adams, K. Carillo, G. L. Mopenza, Toward solving social and technical problems in open source software ecosystems: using cause-and-effect analysis to disentangle the causes of complex problems, IEEE Software 36 (2018) 34–41.

[15] I. Steinmacher, M. A. Graciotto Silva, M. A. Gerosa, D. F. Redmiles, A systematic literature review on the barriers faced by newcomers to open source software projects, Information and Software Technology 59 (2015) 67–85.

[16] F. Fronchetti, I. Wiese, G. Pinto, I. Steinmacher, What Attracts Newcomers to Onboard on OSS Projects? TL;DR: Popularity, in: F. Bordeleau, A. Sillitti, P. Meirelles, V. Lenarduzzi (Eds.), Open Source Systems, Springer International Publishing, Cham, 2019, pp. 91–103. doi:10.1007/978-3-030-20883-7_9.

[17] G. Avelino, E. Constantinou, M. T. Valente, A. Serebrenik, On the abandonment and survival of open source projects: An empirical investigation, in: 2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), IEEE, 2019, pp. 1–12.

[18] I. Steinmacher, I. S. Wiese, T. Conte, M. A. Gerosa, D. Redmiles, The hard life of open source software project newcomers, in: Proceedings of the 7th International Workshop on Cooperative and Human Aspects of Software Engineering, CHASE 2014, Association for Computing Machinery, New York, NY, USA, 2014, pp. 72–78.